

STUDENT TIMETABLE

MANAGEMENT SYSTEM



Name – Saroj Rajpurohit

Registration no. – 26BMR10031

Subject – Introduction to Problem Solving

Guide - Dr. Ashfaq Ahmad Najar

1. Introduction

The Student Timetable Management System is a menu-driven Python program that allows a user to create, view, and analyse a weekly class timetable. The project was developed to apply fundamental algorithms and array/list processing techniques taught in the course to a practical, everyday problem.

2. Objective

- To design a working timetable management tool using a top-down, function-based approach.
- To apply fundamental algorithms such as exchange of values and counting.
- To apply array/list techniques such as finding the maximum, removing duplicates, partitioning, finding the Kth smallest element, and reversal.
- To build a simple, menu-driven interface usable by any student without programming knowledge.

3. Modules Covered

The following table maps the course modules to the concepts used in this project. As instructed, Module 4 (Factoring Methods) was not used since it had not yet been covered in class, and Modules 3 and 5 were given the most emphasis since they were the most recently taught.

Module	Concepts Applied
--------	------------------

Module 1	Top-down design (each feature written as a separate function), program verification through basic self-checks
----------	---

Module 2	Tuples to store class entries, function definitions with parameters, variables and expressions
----------	--

Module 3	Exchange of values (swapping periods), counting, conditional statements and loops for menu control
----------	--

Module 5	Array/list counting, finding maximum, duplicate removal, partitioning, Kth smallest element, array reversal
----------	---

4. System Design

The program stores the timetable as a dictionary where each key is a day of the week and each value is a list of tuples. Each tuple represents one class in the form (subject, start time, end time, class type). This structure allows all array/list-based techniques from Module 5 to be applied directly to the list of classes for each day.

The program follows a top-down design: the main problem (managing a timetable) is broken down into small, independent functions such as add class, check clashes (), busiest day (), and partition theory lab (). A single menu-driven loop in the main () function connects all these functions together, following the control flow concepts from Module 3.

5. Features and Concepts Used

Feature	Concept Used
Add a class	Function with parameters, tuple creation
View timetable (normal / reverse)	Array reversal
Check for clashes	Nested loops, comparison, duplicate detection
Remove duplicate entries	Duplicate removal from a list
Show busiest day	Finding the maximum in a set of values
Subject-wise class count	Array/list counting
Swap two periods	Exchange of values
Partition Theory/Lab classes	Partitioning an array
Kth least busy day	Finding the Kth smallest element
Run verification checks	Program verification

6. Sample Output

A sample run of the program, after adding a few classes:

Monday:

09:00- 10:00 Python Programming (Theory)

10:00-11:00 Engineering Graphics Lab)

Busiest day: Monday with 2 period(s)

No clashes found. Timetable is clean.

7. Conclusion

The Student Timetable Management System successfully demonstrates the practical use of fundamental algorithms and array/list techniques covered in Modules 1, 2, 3, and 5. Features such as clash detection, duplicate removal, finding the busiest day, and partitioning classes show how basic programming concepts can be combined to solve a real, relatable problem faced by students.

8. Tools Used

- Programming Language: Python 3
- Tool: Any Python IDE / command-line interpreter